Highlights

Formalizing Alpha Equivalence for Named and Nameless Lambda Terms

Chris Anto Fröschl, Matthias Güdemann

- Formalizes the proof that common definitions of α-equivalence coincide.
- Formalizes the isomorphism of de Bruijn terms and α-quotiented named terms.
- Connects the named and de Bruijn APIs of CSLib, so results transfer between them.

Formalizing Alpha Equivalence for Named and Nameless Lambda Terms

Chris Anto Fröschl[a,*], Matthias Güdemann[a]

[a] *Munich University of Applied Sciences, Department of Computer Science and Mathematics, Lothstraße 34, Munich, 80335, Bavaria, Germany*

ARTICLE INFO

Keywords:
α-equivalence
permutation action
variable binding
de Bruijn indices
formal verification
proof assistant
Lean

ABSTRACT

Alpha equivalence of λ-terms with named variables is defined either by renaming bound variables or by swapping the bound variables of two abstractions with the same fresh variable, and each style comes in several variants. That these definitions agree is widely accepted, but Crole observed that published proof sketches of this fact are sometimes incorrect. He provided a pen-and-paper proof that several such definitions give the same relation. We formalize Crole's theorem in the Lean 4 proof assistant, as part of the library CSLib. To our knowledge the theorem had not been mechanized before. The formalization supplies the proofs that Crole omits or only sketches. The paper references earlier sources for the now formalized lemmas in question. As a second result we formalize, following Norrish and Vestergaard, an isomorphism between named terms modulo α-equivalence and pure de Bruijn terms, with their respective notions of β-reduction. By the first result it does not matter which of these definitions the quotient is taken by. Through the isomorphism, the Church–Rosser theorem for de Bruijn terms becomes a theorem about named terms, and the two separately developed representations of λ-terms are connected within one library. Both results are part of CSLib, where further work on the untyped λ-calculus can build on them. The statements in this paper are linked to their Lean sources.

1. Introduction

Lambda terms are identified up to the names of their bound variables. For terms with named variables this identification is a relation on terms, i.e. α-equivalence. Several definitions of α-equivalence are found in the literature. The textbook definition renames bound variables: a subterm $\lambda a.\, M$ may be replaced by $\lambda a'.\, M[a := a']$, where a' must not occur in M at all [5, Definition 2.1.11] or must not occur free in it [16, Definition 1.17]. Two terms are α-equivalent if a finite sequence of such replacements turns one into the other. Gabbay and Pitts [13, Proposition 2.2] instead compare two abstractions $\lambda a.\, M$ and $\lambda b.\, N$ directly: they swap both a and b with the same fresh variable z and compare the resulting bodies $(z\ a) \cdot M$ and $(z\ b) \cdot N$. It is widely accepted that these definitions agree. Crole [9] proves in detail that several such relations coincide.

The details are easy to get wrong. In the 1988 edition of their textbook, Hindley and Seldin warned that "the above technicalities are rather dull. But their very dullness has made them a trap for even the most careful authors" (quoted in [9, p. 2]). The current edition only sketches the proofs about α-equivalence, "as they are straightforward and boring" [16, p. 277]. Exercise A1.9 of [16] shows that, contrary to a claim in [10, p. 91], a single α-contraction cannot always be reversed by one step. Crole finds the short proof sketch of Gabbay and Pitts [13, Proposition 2.2] "not quite right", since it lacks a connection between swapping and substitution [9, p. 5]. A formalization that checks every step and stays close to the sources removes such doubts.

Another way to deal with the names of bound variables is to do without them. In de Bruijn's representation [8] a bound variable is a numeral that counts the binders between its occurrence and the binder it refers to. That nameless terms are isomorphic to named terms modulo α-equivalence is stated, for

*Corresponding author
chris.froeschl@hm.edu (C.A. Fröschl); matthias.guedemann@hm.edu (M. Güdemann)
ORCID(s): 0009-0006-3562-5395 (C.A. Fröschl); 0000-0002-1002-6023 (M. Güdemann)

instance, in [5, Appendix C]. Norrish and Vestergaard [21] give a proof mechanized in HOL4 [24], for pure de Bruijn terms with the β-reduction of Nipkow [18].

This paper presents formalizations of both results in the Lean 4 proof assistant [17]. They were developed for CSLib [6], a community-maintained library of computer science for Lean 4 built on Mathlib [25]. CSLib collects reviewed and maintained formalizations of standard results. Both results extend CSLib's named representation of the untyped λ-calculus, which Section 2 recalls.

1. Section 3 formalizes Crole's theorem. Relations on named terms defined by swapping and by a renaming axiom are equal (Theorem 3.4), whereas two further variants of the side condition define different relations (Proposition 3.5). To our knowledge the theorem had not been mechanized before. The formalization follows Crole's proof and supplies the arguments that [9] omits or only sketches (Table 2).
2. Section 4 formalizes the isomorphism of Norrish and Vestergaard between named terms modulo α-equivalence and de Bruijn terms, with their respective notions of β-reduction (Theorem 4.6). The quotient is taken by the α-equivalence of CSLib, which is the first of Crole's relations. As an application, the Church–Rosser theorem for de Bruijn terms yields the Church–Rosser theorem for named terms modulo α-equivalence (Corollary 4.17). This result is not new as a mechanization. We include it to connect the two representations of λ-terms within one library.

The statements of Sections 2 to 4 carry a LEAN symbol that links to the corresponding declaration in the Lean sources. A blueprint[1] presents both formalizations together, with a dependency graph[2] that shows how they are connected.

For Section 3, Aristotle [1], an AI system for automated theorem proving in Lean, was used during the exploration of alternative definitions and auxiliary swap lemmas. The final proofs were substantially rewritten, manually reviewed, and checked against the original paper. A similar verification workflow was used for the formalization in Section 4, where AI assistance was also used to compare the formalization with the existing HOL4 development of Norrish and Vestergaard [21].

Section 5 discusses related work, Section 6 describes the directions we have in mind, in particular the remaining formalizations of the network of Norrish and Vestergaard [21, Figure 1], and Section 7 concludes.

2. Named λ-terms in CSLib

CSLib already provides a named binding representation of λ-terms. We will give the relevant existing notions which will be used in the rest of the paper.

Variables are drawn from an infinite set of atoms, and terms are built from variables, application, and abstraction. Our notation follows Crole [9], except that we write the concrete syntax of λ-terms where Crole writes $P(E_1, E_2)$ and $B([a]E)$ for a generic binary constructor and a generic binder. Crole notes that his proofs are insensitive to this choice [9, Section 2].

Definition 2.1 (Named λ-terms)**.** LEAN Let $\mathbb{A}$ be a set of atoms. The set Λ of named λ-terms is generated by

$$M ::= a \mid M\,M \mid \lambda a.\, M \qquad (a \in \mathbb{A}).$$

We write $\mathrm{occ}(M)$ LEAN for the finite set of all atoms that occur in M, including binding occurrences, and $\mathrm{free}(M)$ LEAN for the finite set of atoms occurring free in M, and abbreviate

$$a \not\triangleright M \iff a \notin \mathrm{occ}(M) \qquad \text{and} \qquad a \# M \iff a \notin \mathrm{free}(M).$$

In the second case we say that a is *fresh* for M. Both notations extend to lists, so that $z \not\triangleright a, b, M, M'$ means that z differs from a and b and occurs in neither M nor M'.

Crole assumes $\mathbb{A}$ to be countable, with equality of atoms as its only operation. In CSLib, $\mathbb{A}$ is an arbitrary type with decidable equality LEAN that comes with a function fresh LEAN from finite sets of atoms to atoms such that $\mathrm{fresh}(S) \notin S$. In particular $\mathbb{A}$ is infinite, and an atom avoiding any given finite set can be computed.

Three operations act on the atoms of a term. The first is purely textual and is the one CSLib's default α-equivalence is defined with.

[1] https://chris-anto-froeschl.github.io/untyped-lambda-calculus/
[2] https://chris-anto-froeschl.github.io/untyped-lambda-calculus/blueprint/dep_graph_document.html

Definition 2.2 (Renaming). LEAN $M\langle a \mapsto b\rangle$ is M with every occurrence of a, free, bound or binding, replaced by b.

The second applies a bijection on atoms to every atom of a term, binding occurrences included.

Definition 2.3 (Permutation action [9, Section 2]). LEAN For a permutation π of $\mathbb{A}$, the term $\pi \cdot M$ is defined by

$$\pi \cdot a = \pi(a) \qquad \pi \cdot (M_1\, M_2) = (\pi \cdot M_1)\,(\pi \cdot M_2) \qquad \pi \cdot (\lambda a.\, M) = \lambda \pi(a).\, \pi \cdot M.$$

For the transposition $(a\ b)$ we call $(a\ b) \cdot M$ the *swap* LEAN of a and b in M. The *agreement set* of two permutations is $\mathrm{AS}(\pi, \pi') = \{\, a \in \mathbb{A} \mid \pi(a) = \pi'(a) \,\}$.

Unlike renaming (Definition 2.2), a swap is a bijection on terms and never identifies two atoms. The two operations agree when the new atom is unused: if $z \notin M$ then $(z\ a) \cdot M = M\langle a \mapsto z\rangle$ LEAN.

The third is capture-avoiding substitution, which Crole writes $E\{a'/a\}$ for the substitution of an atom and which we write $M[a := N]$ for the substitution of an arbitrary term.

Definition 2.4 (Capture-avoiding substitution). LEAN

$$b[a := N] = \begin{cases} N & \text{if } b = a \\ b & \text{otherwise} \end{cases}$$

$$(M_1\, M_2)[a := N] = M_1[a := N]\ M_2[a := N]$$

$$(\lambda b.\, M)[a := N] = \begin{cases} \lambda b.\, M & \text{if } b = a \\ \lambda b.\, M[a := N] & \text{if } b \neq a \text{ and } b \# N \\ \lambda c.\, M\langle b \mapsto c\rangle[a := N] & \text{otherwise, where } c = \mathrm{fresh}(\mathrm{occ}(M) \cup \mathrm{occ}(N) \cup \{a, b\}) \end{cases}$$

Example 2.5 (The three operations). Let $M = \lambda a.\, a\, b$ for distinct atoms a and b.

- $M\langle a \mapsto b\rangle = \lambda b.\, b\, b$ LEAN: renaming identifies the two atoms, so that the free b is captured.
- $(a\ b) \cdot M = \lambda b.\, b\, a$ LEAN: the swap exchanges them.
- $M[b := a] = \lambda c.\, c\, a$ for a fresh atom c LEAN: substitution replaces only the free occurrence of b, and renames the binder to avoid capturing a.

Definition 2.4 coincides with Gabbay's [11, Definition 5.13]. This definition is shown to be well defined in [11, Lemma 5.14].

Renaming, substitution, a notion of α-equivalence (Definition 3.1 below) and its basic theory, including the substitution lemma [5, Lemma 2.1.16] LEAN, are due to Yin and Montesi[3]. We take the existing α-equivalence of CSLib as the starting point of our work against which the follow-up Section 3 compares the other definitions.

3. Characterizations of α-equivalence

This section presents Crole's result [9] as it is formalized in CSLib. We keep Crole's names for the relations, the rules and the side conditions, and each statement references its counterpart in [9].

3.1. The relations

The first three relations are syntax-directed and differ only in the side condition on the atom z that replaces the two bound atoms.

[3] https://github.com/leanprover/cslib/pull/458

Definition 3.1 ([9, Definitions 3.1–3.3]). The relation $\sim_{\mathrm{p}}$ LEMMA on Λ is inductively defined by

$$\frac{}{a \sim_{\mathrm{p}} a}\ \textsc{atom} \qquad \frac{M_1 \sim_{\mathrm{p}} M_1' \quad M_2 \sim_{\mathrm{p}} M_2'}{M_1\, M_2 \sim_{\mathrm{p}} M_1'\, M_2'}\ \textsc{pcg} \qquad \frac{(z\ a) \cdot M \sim_{\mathrm{p}} (z\ b) \cdot M'}{\lambda a.\, M \sim_{\mathrm{p}} \lambda b.\, M'}\ \textsc{pi}\ [z \not\in a, b, M, M'].$$

The relations $\sim_{\mathrm{p}\#}$ LEMMA and $\sim_{\mathrm{p}}^{1}$ LEMMA are defined by the same rules, with the side condition of PI replaced by

$$[z \# a, b, M, M'] \qquad \text{and} \qquad [z \not\in M, M'],$$

respectively. The resulting rules are called PI# and PI1 respectively.

Read as rules on λ-terms, ATOM and PCG compare variables and applications component by component, and PI compares two abstractions by moving both bound atoms to a common unused atom z and comparing the bodies.

The relation $\sim_{\mathrm{p}}$ follows Gabbay and Pitts [13] and is the α-equivalence $=_\alpha$ that CSLib already provides. The remaining two relations are generated by the renaming axiom and are congruences by definition.

Definition 3.2 ([9, Definitions 3.4 and 3.5]). The relation $\sim_{\mathrm{r}}$ LEMMA on Λ is inductively defined by

$$\frac{}{M \sim_{\mathrm{r}} M}\ \textsc{ref} \qquad \frac{M_1 \sim_{\mathrm{r}} M_2}{M_2 \sim_{\mathrm{r}} M_1}\ \textsc{sym} \qquad \frac{M_1 \sim_{\mathrm{r}} M_2 \quad M_2 \sim_{\mathrm{r}} M_3}{M_1 \sim_{\mathrm{r}} M_3}\ \textsc{trs}$$

$$\frac{M_1 \sim_{\mathrm{r}} M_1' \quad M_2 \sim_{\mathrm{r}} M_2'}{M_1\, M_2 \sim_{\mathrm{r}} M_1'\, M_2'}\ \textsc{pcg} \qquad \frac{M \sim_{\mathrm{r}} M'}{\lambda a.\, M \sim_{\mathrm{r}} \lambda a.\, M'}\ \textsc{bcg} \qquad \frac{}{\lambda a.\, M \sim_{\mathrm{r}} \lambda a'.\, M[a := a']}\ \alpha\ [a' \not\in a, M].$$

The relation $\sim_{\mathrm{r}\#}$ LEMMA is defined by the same rules, with the side condition of α replaced by $[a' \# a, M]$. The resulting rule is called $\alpha\#$.

The relation $\sim_{\mathrm{r}}$ corresponds to the congruence $\equiv_\alpha$ of Barendregt [5, Definition 2.1.11], and $\sim_{\mathrm{r}\#}$ to that of Hindley and Seldin [16, Definition 1.17].

Example 3.3 (Running example). For distinct atoms a, b, c, the terms $\lambda a.\, a\, b$ and $\lambda c.\, c\, b$ are related by all of these relations, each time by a single application of the abstraction rule.

- PI LEMMA, PI# LEMMA and PI1 LEMMA: any witness d distinct from a, b, c meets all three side conditions, and $(d\ a) \cdot (a\ b) = d\, b = (d\ c) \cdot (c\ b)$.
- α LEMMA and $\alpha\#$ LEMMA: rename a to c, which does not occur in $a\, b$. Then $(a\, b)[a := c] = c\, b$.

3.2. The characterization theorem

Theorem 3.4 (Crole [9, Theorems 4.1, 4.2, 4.4–4.6]). *For all terms $M, N \in \Lambda$ the following are equivalent:*

(i) $M \sim_{\mathrm{p}} N$,
(ii) $M \sim_{\mathrm{p}\#} N$ LEMMA,
(iii) $M \sim_{\mathrm{p}}^{1} N$ LEMMA,
(iv) $M \sim_{\mathrm{r}} N$ LEMMA,
(v) $M \sim_{\mathrm{r}\#} N$ LEMMA.

The proof is outlined in Section 3.3. The side condition of PI cannot be weakened much further.

Proposition 3.5 ([9, Proposition 4.3]). LEMMA *Let $\sim_{\mathrm{p}}^{2}$ LEMMA and $\sim_{\mathrm{p}}^{3}$ LEMMA be defined as $\sim_{\mathrm{p}}$, with the side condition of PI replaced by $[z \not\in a, b]$ and by no condition, respectively. Then $\sim_{\mathrm{p}}^{2} \neq \sim_{\mathrm{p}}$ and $\sim_{\mathrm{p}}^{3} \neq \sim_{\mathrm{p}}$.*

Proof. For distinct atoms a, b, z we have $\lambda a.\, a\, z \sim_{\mathrm{p}}^{2} \lambda b.\, b\, a$ LEMMA with witness z, and $\lambda a.\, b\, a \sim_{\mathrm{p}}^{3} \lambda b.\, a\, b$ LEMMA with witness b. In both cases the two terms have different free atoms. Since $\sim_{\mathrm{p}}$ preserves free atoms LEMMA, they are not related by $\sim_{\mathrm{p}}$ LEMMA LEMMA. □

3.3. The proof

Since $\sim_{\mathrm{p}}$ has no rules for reflexivity, symmetry and transitivity, these have to be proved. CSLib already provides them LEAN LEAN LEAN, together with two facts used below: related terms have the same free atoms LEAN, and substitution respects $\sim_{\mathrm{p}}$: if $M \sim_{\mathrm{p}} M'$ and $N \sim_{\mathrm{p}} N'$ then $M[a := N] \sim_{\mathrm{p}} M'[a := N']$ LEAN. Beyond these, the proof of Theorem 3.4 rests on three facts: $\sim_{\mathrm{p}}$ is preserved by swapping, a permutation acts up to $\sim_{\mathrm{p}}$ only through the free atoms, and swapping with an unused atom is, up to α-equivalence, the same as substitution.

Lemma 3.6 ([9, Lemma 6.1]). LEAN *If $M \sim_{\mathrm{p}} M'$ then $(u\ v) \cdot M \sim_{\mathrm{p}} (u\ v) \cdot M'$, for all atoms u, v.*

This is an instance of the equivariance principle of nominal techniques [11, Theorem 4.4], which will again appear in Section 4. For the rule PI, each of the atoms a, b, z of the rule is equal to u, equal to v, or different from both, which gives $3^3 = 27$ cases. The side condition $z \neq a, b$ excludes 10 of them. In the 9 cases with $z \neq u, v$ the swap $(u\ v)$ commutes with $(z\ a)$ and $(z\ b)$, and z is again a witness. Crole works out one of these and three of the remaining eight [9, pp. 14–15]. The formalization derives the other five by exchanging u and v or the two sides of $\sim_{\mathrm{p}}$ (Table 1).

Lemma 3.7 ([9, Lemma 6.2]). *Let π, π' be permutations of $\mathbb{A}$ and M a term.*

1. LEAN *If $\mathrm{occ}(M) \subseteq \mathrm{AS}(\pi, \pi')$ then $\pi \cdot M = \pi' \cdot M$.*
2. LEAN *If $\mathrm{free}(M) \subseteq \mathrm{AS}(\pi, \pi')$ then $\pi \cdot M \sim_{\mathrm{p}} \pi' \cdot M$.*

A permutation thus acts on a term only through the atoms of the term, and up to α-equivalence only through its free atoms.

Example 3.8. Let $M = \lambda a.\, a\, b$, $\pi = (a\ c)$ and $\pi' = \mathrm{id}$ with $c \neq b$. Part 1 of Lemma 3.7 does not apply, since π and π' disagree on $a \in \mathrm{occ}(M)$, and indeed $\pi \cdot M = \lambda c.\, c\, b$ is a different term from M. But $\mathrm{free}(M) = \{b\} \subseteq \mathrm{AS}(\pi, \pi')$, so part 2 gives $\lambda c.\, c\, b \sim_{\mathrm{p}} \lambda a.\, a\, b$, the running example of Example 3.3 LEAN.

The third fact links the two families of relations. Crole points out that the proof sketch of Gabbay and Pitts [13, Proposition 2.2] lacks it: they argue only that swapping preserves $\sim_{\mathrm{r}}$, which gives no connection between swapping and substitution [9, p. 5].

Lemma 3.9 ([9, Lemmas 6.3 and 6.4]). *If $z \not\sharp M$ then $(z\ a) \cdot M \sim_{\mathrm{r}} M[a := z]$* LEAN *and $(z\ a) \cdot M \sim_{\mathrm{r}\#} M[a := z]$* LEAN.

Example 3.10. The two sides of Lemma 3.9 differ on bound occurrences of a, which the swap replaces and the substitution does not. Let $M = a\,(\lambda a.\, a)$ and $z \neq a$. Then

$$(z\ a) \cdot M = z\,(\lambda z.\, z) \text{ LEAN} \qquad \text{and} \qquad M[a := z] = z\,(\lambda a.\, a) \text{ LEAN},$$

and the two are related by PCG, α and SYM LEAN.

For $\sim_{\mathrm{p}}$ the corresponding statement compares two swaps. We need only the case $n = 1$ of Crole's lemma.

Table 1
The cases of the rule PI in the proof of Lemma 3.6, by the values of a, b, z (o: neither u nor v). Entries 1–4 are the cases Crole works out [9, pp. 14–15], each with the new witness for PI: 1 is (o, o, o) with witness z LEAN, 2 is (o, o, u) with witness v LEAN, 3 is (o, u, v) with witness u LEAN, and 4 is (u, u, v) with witness u LEAN. A superscript σ means that the case follows by exchanging u and v, and τ by exchanging the two sides of $\sim_{\mathrm{p}}$ (symmetry). The symbol $\times$ marks the cases excluded by $z \neq a, b$. The assignment is the one of the Lean proof LEAN.

	$z = u$			$z = v$			$z = o$		
	$b = u$	$b = v$	$b = o$	$b = u$	$b = v$	$b = o$	$b = u$	$b = v$	$b = o$
$a = u$	$\times$	$\times$	$\times$	4	$\times$	3^{τ}	1	1	1
$a = v$	$\times$	4^{σ}	$3^{\sigma\tau}$	$\times$	$\times$	$\times$	1	1	1
$a = o$	$\times$	3^{σ}	2	3	$\times$	2^{σ}	1	1	1

Lemma 3.11 ([9, Lemma 6.5], case $n = 1$)**.** LEAN *Let z, a, a' be distinct atoms and M a term with $z \# M$ and $a' \# M$. Then $(z\ a) \cdot M \sim_{\mathrm{p}} (z\ a') \cdot M[a := a']$.*

Proof idea. Crole omits the proof. First $M[a := a'] \sim_{\mathrm{p}} (a\ a') \cdot M$ LEAN: rename the bound occurrences of a' away [9, Lemma 6.6], use that substitution respects $\sim_{\mathrm{p}}$, and apply the $\sim_{\mathrm{p}}$-form of Lemma 3.9 LEAN. Then $(z\ a)$ and $(z\ a') \circ (a\ a')$ agree on free(M), and Lemma 3.7 applies LEAN. □

The formalization also proves Crole's general form, for n swaps applied one after the other, by an induction on n that commutes the remaining swaps past the first.

Proof idea of Theorem 3.4. Each of (ii)–(v) is shown equivalent to (i) by showing that each of the two relations is closed under the rules of the other. Only the rules for abstractions need an argument, and except for the rules α and $\alpha\#$ the argument follows Crole [9, pp. 5–8]. A witness that is only fresh (PI#) or may equal a bound atom (PI1) is exchanged for an unused one by Lemmas 3.6 and 3.7. The renaming relations are closed under PI: Lemma 3.9 turns the two swaps of its premise into substitutions, and two instances of α or $\alpha\#$ complete the step.

For the converse, reflexivity, symmetry and transitivity of $\sim_{\mathrm{p}}$ are already in CSLib, and BCG follows from Lemma 3.6 and PI. For α and $\alpha\#$, let $a' \# M$ and choose $z \not\in a, a', M, M[a := a']$. Then $(z\ a) \cdot M \sim_{\mathrm{p}} (z\ a') \cdot M[a := a']$ by Lemma 3.11, which is the premise of PI for $\lambda a.\, M \sim_{\mathrm{p}} \lambda a'.\, M[a := a']$. □

3.4. Notes on the formalization

The formalization follows [9] closely, and we found no errors in it. Our additions to CSLib are the four relations other than $\sim_{\mathrm{p}}$, Theorem 3.4, Proposition 3.5, Lemmas 3.6–3.11 and Crole's Lemma 6.6. The rest belongs to the named representation of Section 2.

Where the formalization departs from [9] beyond the proofs discussed above:

- *Order of the results.* Reflexivity, symmetry and transitivity of $\sim_{\mathrm{p}}$ are proved directly. Crole obtains them inside the proof of his Theorem 4.4 [9, p. 6] and cites that theorem for them in the proof of Theorem 4.1 [9, p. 5]. In the formalization Theorem 4.1 does not depend on Theorem 4.4.
- *Renaming in PI.* The rules PI and PI1 are stated in CSLib with renaming (Definition 2.2) in place of the swap. Under their side conditions the two coincide.
- *The rules α and $\alpha\#$.* Crole proves α by an induction on M [9, pp. 6–7]. For $\alpha\#$ this induction fails, because $M[a := a']$ may have to rename a binder, and Crole introduces Lemma 6.5 for general n to repair it [9, p. 8]. The formalization derives both rules from the case $n = 1$ (Lemma 3.11).
- *Freshness in Lemma 6.5.* Crole assumes $z \not\in M$, but freshness suffices. A bound occurrence of z can first be renamed away by Crole's Lemma 6.6: if $a \# M$ then there is a term $\hat{M}$ with $a \not\in \hat{M}$ and $\hat{M} \sim_{\mathrm{r}} M$ LEAN. Crole regards this lemma as a form of Barendregt's variable convention [5, Convention 2.1.13], [9, p. 8].
- *Lemma 6.6 for $\sim_{\mathrm{p}}$.* Crole states Lemma 6.6 for $\sim_{\mathrm{r}}$ only. Lemma 3.11 needs it for $\sim_{\mathrm{p}}$ LEAN. This form cannot be derived from Theorem 3.4, because that theorem's proof uses Lemma 3.11. The proof follows Crole's sketch [9, p. 16], except where the binder is a itself. There, renaming a to an unused atom throughout the body removes every occurrence at once, so no induction on the size of the term is needed.
- *Permutations.* Lemma 3.7 is proved for arbitrary bijections on $\mathbb{A}$, where Crole assumes finite support, a difference that is immaterial for terms [11, Remark 2.25]. Agreement on a set of atoms is stated with Mathlib's `Set.EqOn` LEAN instead of a dedicated agreement set definition.
- *Scope.* Crole's second contribution, α-equivalence of program contexts [9, Section 5], is not covered.

What a formalization has to add to [9] are the proofs that the paper leaves to the reader or to the literature. Table 2 lists the lemmas of [9, Section 6.1] that the characterization needs, how much of their proofs Crole gives, and where the statement or its argument appears earlier in the literature. Lemma 6.7, which Crole uses only for a second, direct proof of Theorem 4.6, is not needed and not formalized. Crole names a source only for Lemma 6.3. The other entries in the last column are ours. None of the omitted proofs held a surprise.

Table 2
The lemmas of [9, Section 6.1]: their proofs in [9] and earlier sources.

In [9]	Proof there	Earlier source
Lemma 6.1	representative cases	[13, proof of Proposition 2.2]; [26, (9)]; [11, Theorem 4.4]
Lemma 6.2, part 1	omitted	[13, Example 3.5(ii)]
Lemma 6.2, part 2	given	[26, Lemma 2.8]; [12, Lemma 3.8]
Lemma 6.3	given	[11, Lemma 5.16]; [12, Lemma 3.6(2)]
Lemma 6.4	as for Lemma 6.3	as for Lemma 6.3
Lemma 6.5	details omitted	none known to us for general n. For $n = 1$, the first step of our argument follows [16, Lemmas A1.7, A1.8]
Lemma 6.6	sketched, well known	[16, Lemma A1.5(g)]

4. An isomorphism with de Bruijn terms

De Bruijn indices remove the need for α-equivalence altogether by replacing bound variable names with numerals counting binder depth. Norrish and Vestergaard [21] give a direct, mechanized proof that the pure de Bruijn calculus is isomorphic to the α-quotiented named calculus, so that a result proved on de Bruijn terms is a result about λ-terms in the ordinary sense. We formalize this isomorphism for the named terms and the de Bruijn terms of CSLib; the formalization is under review for inclusion in CSLib[4], on top of the pull requests for the de Bruijn terms (below) and for Theorem 4.1 of Section 3. By Theorem 3.4 it does not matter which of these α-equivalences is quotiented by; the formalization uses $\sim_{\mathrm{p}}$.

Norrish and Vestergaard place their isomorphism in a web of directly proved relationships between ten formalizations of the untyped λ-calculus, reproduced in Figure 1. With this section, CSLib contains the nodes dB, dB′ and λ^N_α and the two dashed edges between them.

We keep the notation of [21] on the de Bruijn side, where t, u, s range over de Bruijn terms and i, j, k, n over indices. As in [21], M and N range over α-equivalence classes in this section.

4.1. De Bruijn terms

The de Bruijn terms, their substitution and β-reduction, and the proof of the Church–Rosser theorem used in Section 4.7 are not ours: they are due to Wang, the syntax and substitution jointly with Radziwill[5].

Definition 4.1 (De Bruijn terms [21, Section 2.2]). LEAN The set dB of de Bruijn terms is generated by

$$t ::= \mathsf{dV}\, i \mid \mathsf{dAPP}\, t\, t \mid \mathsf{dABS}\, t \qquad (i \in \mathbb{N}).$$

Lifting LEAN and substitution LEAN are defined as in Nipkow's formalization [18]:

$$\begin{aligned}
\mathsf{lift}\,(\mathsf{dV}\, i)\, k &= \text{if } i < k \text{ then } \mathsf{dV}\, i \text{ else } \mathsf{dV}\,(i+1)\\
\mathsf{lift}\,(\mathsf{dAPP}\, t\, u)\, k &= \mathsf{dAPP}\,(\mathsf{lift}\, t\, k)\,(\mathsf{lift}\, u\, k)\\
\mathsf{lift}\,(\mathsf{dABS}\, t)\, k &= \mathsf{dABS}\,(\mathsf{lift}\, t\,(k+1))
\end{aligned}$$

$$\begin{aligned}
\mathsf{nsub}\, s\, k\,(\mathsf{dV}\, i) &= \text{if } k < i \text{ then } \mathsf{dV}\,(i-1) \text{ else if } i = k \text{ then } s \text{ else } \mathsf{dV}\, i\\
\mathsf{nsub}\, s\, k\,(\mathsf{dAPP}\, t\, u) &= \mathsf{dAPP}\,(\mathsf{nsub}\, s\, k\, t)\,(\mathsf{nsub}\, s\, k\, u)\\
\mathsf{nsub}\, s\, k\,(\mathsf{dABS}\, t) &= \mathsf{dABS}\,(\mathsf{nsub}\,(\mathsf{lift}\, s\, 0)\,(k+1)\, t)
\end{aligned}$$

The function nsub substitutes for the index k and at the same time decrements all larger free indices, which is what β-contraction needs when a binder disappears.

Definition 4.2 ([21, Definition 1]). LEAN The relation $\to_{\mathrm{d}}$ on dB is the least relation satisfying

$$\frac{}{\mathsf{dAPP}\,(\mathsf{dABS}\, t)\, u \to_{\mathrm{d}} \mathsf{nsub}\, u\, 0\, t} \qquad \frac{t \to_{\mathrm{d}} u}{\mathsf{dABS}\, t \to_{\mathrm{d}} \mathsf{dABS}\, u}$$

[4] https://github.com/leanprover/cslib/pull/TODO
[5] https://github.com/leanprover/cslib/pull/475

$$\frac{t \to_{\mathrm{d}} u}{\mathsf{dAPP}\, t\, s \to_{\mathrm{d}} \mathsf{dAPP}\, u\, s} \qquad \frac{t \to_{\mathrm{d}} u}{\mathsf{dAPP}\, s\, t \to_{\mathrm{d}} \mathsf{dAPP}\, s\, u}\,.$$

4.2. The α-quotiented calculus

Definition 4.3 ([21, Section 2.1]). LEAN Λ_α is the quotient of Λ by $\sim_{\mathrm{p}}$. We write $[M]_\alpha$ for the class of a term $M \in \Lambda$. Application, abstraction and substitution LEAN are lifted from Λ to Λ_α:

$$[M]_\alpha\,[N]_\alpha = [M\,N]_\alpha \qquad \lambda a.\,[M]_\alpha = [\lambda a.\,M]_\alpha \qquad [M]_\alpha[a := [N]_\alpha] = [M[a := N]]_\alpha.$$

These operations are well defined because $\sim_{\mathrm{p}}$ is a congruence and is preserved by substitution LEAN. Abstraction on Λ_α is not injective (Example 4.4), so Λ_α is not a free algebra.

Example 4.4. LEAN For distinct atoms a, b, c, $\lambda a.\,[a\,b]_\alpha = \lambda c.\,[c\,b]_\alpha$ (Example 3.3), although $[a\,b]_\alpha \neq [c\,b]_\alpha$.

Definition 4.5 (β-reduction on Λ_α). LEAN The relation $\to_\beta$ on Λ_α is the least relation satisfying

$$\frac{}{(\lambda a.\,M)\,N \to_\beta M[a := N]} \qquad \frac{M \to_\beta N}{\lambda a.\,M \to_\beta \lambda a.\,N}$$

$$\frac{M \to_\beta N}{M\,Z \to_\beta N\,Z} \qquad \frac{M \to_\beta N}{Z\,M \to_\beta Z\,N}\,.$$

This is the system $\lambda^{\mathrm{N}}_\alpha$ of [21] up to the definition of substitution. There, substitution is defined directly on the quotient by a recursion principle [20, Section 2.1]. Here, it is lifted from raw terms, which is how Norrish and Vestergaard describe the construction of λ^{Ho}_α [21, Section 2.1].

Two systems $\langle X, \to_X\rangle$ and $\langle Y, \to_Y\rangle$ are *isomorphic* if there is a bijection $f\colon X \to Y$ with $x_1 \to_X x_2 \iff f(x_1) \to_Y f(x_2)$ for all x_1, x_2 [21, Section 2.3].

4.3. The isomorphism theorem

Theorem 4.6 ([21, Theorem 20]). LEAN *The systems $\langle \Lambda_\alpha, \to_\beta\rangle$ and $\langle \mathrm{dB}, \to_{\mathrm{d}}\rangle$ are isomorphic, as witnessed by the bijection f of Theorem 4.10 below: for all $M, N \in \Lambda_\alpha$,*

$$M \to_\beta N \iff f(M) \to_{\mathrm{d}} f(N).$$

Proof. From Theorems 4.12 and 4.15 below. □

The notion of isomorphic reduction systems used here is exactly Mathlib's relation isomorphism, and the formalization also states Theorem 4.6 in that form, $\langle \Lambda_\alpha, \to_\beta\rangle \simeq_r \langle \mathrm{dB}, \to_{\mathrm{d}}\rangle$ LEAN.

The proof of Norrish and Vestergaard goes through an intermediate formalism (Figure 2). Section 4.4 gives a second set of rules on the de Bruijn terms that is built to resemble Λ_α. Section 4.5 constructs f and shows that it turns $\to_\beta$ into these rules, and Section 4.6 shows that they define the same relation as $\to_{\mathrm{d}}$.

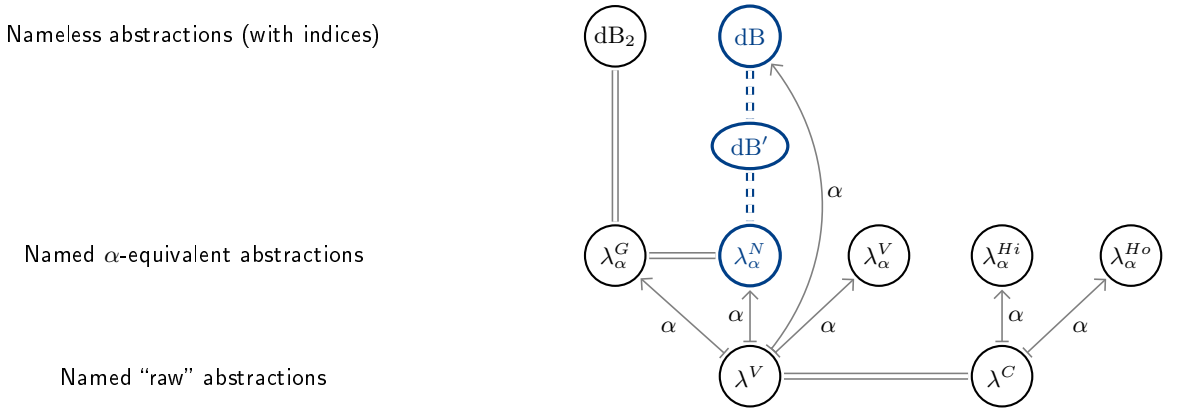


Figure 1: Formalizations of the untyped λ-calculus and their directly proved relationships, redrawn from [21, Figure 1] with the help of Claude (Anthropic) and checked against the original. Double lines are isomorphisms and arrows are α-quotients. The dashed lines are the results of [21], which this section formalizes; thick blue outlines mark what is formalized in CSLib.

4.4. An intermediate formalism

The proof goes through a second presentation dB' of the same type dB that is built to resemble Λ_α. It replaces nsub by a substitution that leaves the other indices alone, and dABS by an abstraction over a chosen index.

Definition 4.7 ([21, Definitions 4 and 5]). LEMMA LEMMA

$$\begin{aligned}
\mathsf{sub}\, s\, k\, (\mathsf{dV}\, i) &= \text{if } i = k \text{ then } s \text{ else } \mathsf{dV}\, i \\
\mathsf{sub}\, s\, k\, (\mathsf{dAPP}\, t\, u) &= \mathsf{dAPP}\, (\mathsf{sub}\, s\, k\, t)\, (\mathsf{sub}\, s\, k\, u) \\
\mathsf{sub}\, s\, k\, (\mathsf{dABS}\, t) &= \mathsf{dABS}\, (\mathsf{sub}\, (\mathsf{lift}\, s\, 0)\, (k+1)\, t) \\
\mathsf{dLAM}\, i\, t &= \mathsf{dABS}\, (\mathsf{sub}\, (\mathsf{dV}\, 0)\, (i+1)\, (\mathsf{lift}\, t\, 0))
\end{aligned}$$

Like abstraction on Λ_α, dLAM is not injective (Example 4.8). Every dABS term is a dLAM term over any index that is not free in it LEMMA [21, Lemma 6].

Example 4.8. LEMMA For every index i,

$$\mathsf{dLAM}\, i\, (\mathsf{dV}\, i) = \mathsf{dABS}\, (\mathsf{sub}\, (\mathsf{dV}\, 0)\, (i+1)\, (\mathsf{dV}\, (i+1))) = \mathsf{dABS}\, (\mathsf{dV}\, 0),$$

the de Bruijn form of $\lambda a.\, a$ for every atom a.

Definition 4.9 ([21, Definition 7]). LEMMA The relation $\to_{\mathrm{d}'}$ on dB is the least relation satisfying

$$\frac{}{\mathsf{dAPP}\, (\mathsf{dLAM}\, i\, t)\, u \to_{\mathrm{d}'} \mathsf{sub}\, u\, i\, t} \qquad \frac{t \to_{\mathrm{d}'} u}{\mathsf{dLAM}\, i\, t \to_{\mathrm{d}'} \mathsf{dLAM}\, i\, u}$$

$$\frac{t \to_{\mathrm{d}'} u}{\mathsf{dAPP}\, t\, s \to_{\mathrm{d}'} \mathsf{dAPP}\, u\, s} \qquad \frac{t \to_{\mathrm{d}'} u}{\mathsf{dAPP}\, s\, t \to_{\mathrm{d}'} \mathsf{dAPP}\, s\, u}\,.$$

4.5. The bijection

Fix a bijection between $\mathbb{A}$ and $\mathbb{N}$, written with a hat in both directions as in [21]: $\hat{a}$ is the number of the atom a and $\hat{\imath}$ the atom with number i.

Theorem 4.10 ([21, Theorem 11]). LEMMA *There is a bijection* $f\colon \Lambda_\alpha \to \mathrm{dB}$ *with*

$$f([a]_\alpha) = \mathsf{dV}\, \hat{a} \qquad f(M\, N) = \mathsf{dAPP}\, (f\, M)\, (f\, N) \qquad f(\lambda a.\, M) = \mathsf{dLAM}\, \hat{a}\, (f\, M). \tag{1}$$

Example 4.11 (Running example, continued). LEMMA For distinct atoms a and b,

$$f(\lambda a.\, a\, b) = \mathsf{dLAM}\, \hat{a}\, (\mathsf{dAPP}\, (\mathsf{dV}\, \hat{a})\, (\mathsf{dV}\, \hat{b})) = \mathsf{dABS}\, (\mathsf{dAPP}\, (\mathsf{dV}\, 0)\, (\mathsf{dV}\, (\hat{b}+1)))\,:$$

the bound atom becomes the index 0, and the free atom b is shifted past the binder. For every atom $c \neq b$, the α-equivalent term $\lambda c.\, c\, b$ has the same image LEMMA.

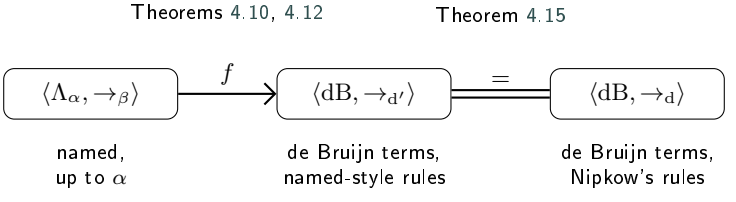


Figure 2: The route of the proof of Theorem 4.6.

Since Λ_α is not a free algebra, the equations (1) do not define a function by themselves. Norrish and Vestergaard obtain f from Pitts' recursion principle for Λ_α [22], for which dB has to be shown to be a nominal set. The formalization takes a more elementary route: the equations (1) define a function on raw terms by structural recursion, this function is shown to respect $\sim_\mathrm{p}$, and f is its lift to the quotient. Injectivity is proved by structural induction on raw terms. Its abstraction case rests on the fact that dLAM is injective up to a transposition: $\mathsf{dLAM}\,i\,t = \mathsf{dLAM}\,j\,u$ holds exactly if $i = j$ and $t = u$, or $i \neq j$, the index i is not free in u, and t is u with i and j exchanged. Surjectivity is proved, as in [21], by complete induction on the size of the de Bruijn term, using Lemma 6 of [21] for dABS terms.

Theorem 4.12 ([21, Theorem 12])**.** *For all $M, N \in \Lambda_\alpha$: $M \to_\beta N$ if and only if $f(M) \to_{\mathrm{d}'} f(N)$.*

Proof idea. The bijection turns substitution on Λ_α into the index-preserving substitution sub, $f(M[a := N]) = \mathsf{sub}\,(f\,N)\,\hat{a}\,(f\,M)$, and abstraction into dLAM, so that each rule of $\to_\beta$ becomes the corresponding rule of $\to_{\mathrm{d}'}$ [21, proof of Theorem 12]. □

4.6. From $\to_{\mathrm{d}'}$ to $\to_\mathrm{d}$

It remains to show that $\to_\mathrm{d}$ and $\to_{\mathrm{d}'}$ are the same relation on dB. Their rules for redexes are related by an identity between sub and nsub [21, Lemma 13]; the difficulty is to match the congruence rule for dABS with the one for dLAM. This is where permutations enter on the de Bruijn side. A permutation π of atoms acts on a de Bruijn term through the free indices:

$$\pi \cdot (\mathsf{dV}\,i) = \mathsf{dV}\,\widehat{\pi(i)} \qquad \pi \cdot (\mathsf{dAPP}\,t\,u) = \mathsf{dAPP}\,(\pi \cdot t)\,(\pi \cdot u) \qquad \pi \cdot (\mathsf{dABS}\,t) = \mathsf{dABS}\,(\mathsf{inc}(\pi) \cdot t),$$

where $\mathsf{inc}(\pi)$ is π shifted by one index, so that it fixes the index bound by the abstraction [21, Definition 8].

Example 4.13. Let $\hat{a} = 0$ and $\hat{b} = 1$. The term $\mathsf{dABS}\,(\mathsf{dV}\,1)$ is $\lambda x.\,a$, since index 1 under one binder is the free index 0. Under the binder, $\mathsf{inc}(a\;b)$ exchanges the indices 1 and 2, so

$$(a\;b) \cdot \mathsf{dABS}\,(\mathsf{dV}\,1) = \mathsf{dABS}\,(\mathsf{dV}\,2),$$

which is $\lambda x.\,b$. Exchanging 0 and 1 under the binder instead would give $\mathsf{dABS}\,(\mathsf{dV}\,0)$, which is $\lambda x.\,x$.

A relation R on dB is *equivariant* if $R\,(\pi \cdot t)\,(\pi \cdot u) \iff R\,t\,u$ for all π.

By [21, Theorem 10], sub and dLAM commute with permutations, which act on their index arguments j as $\widehat{\pi(j)}$, and hence $\to_{\mathrm{d}'}$ is equivariant.

The key observation of [21] is that lifting is itself the action of a permutation [21, Lemma 14], and so is the substitution of a fresh index. A dABS term and its presentation as a dLAM term therefore differ by a permutation, which an equivariant relation cannot see.

Theorem 4.14 ([21, Theorem 15])**.** *Let R be an equivariant relation on* dB*. Then*

$$\big(\forall t\,u.\; R\,t\,u \Rightarrow R\,(\mathsf{dABS}\,t)\,(\mathsf{dABS}\,u)\big) \iff \big(\forall i\,t\,u.\; R\,t\,u \Rightarrow R\,(\mathsf{dLAM}\,i\,t)\,(\mathsf{dLAM}\,i\,u)\big).$$

Theorem 4.14 applies to $\to_{\mathrm{d}'}$, which is equivariant. To apply it to $\to_\mathrm{d}$ one needs equivariance of $\to_\mathrm{d}$, and hence the behavior of nsub under a permutation [21, Lemma 17].

Theorem 4.15 ([21, Theorem 19])**.** *For all $t, u \in$ dB: $t \to_{\mathrm{d}'} u$ if and only if $t \to_\mathrm{d} u$.*

Example 4.16 (Running example, continued)**.** For distinct atoms a, b, c, the step $(\lambda a.\,[a\,b]_\alpha)\,[c]_\alpha \to_\beta [c\,b]_\alpha$ becomes

$$\mathsf{dAPP}\,(\mathsf{dABS}\,(\mathsf{dAPP}\,(\mathsf{dV}\,0)\,(\mathsf{dV}\,(\hat{b}+1))))\,(\mathsf{dV}\,\hat{c}) \;\to_\mathrm{d}\; \mathsf{dAPP}\,(\mathsf{dV}\,\hat{c})\,(\mathsf{dV}\,\hat{b})$$

: nsub replaces the bound index 0 by $\mathsf{dV}\,\hat{c}$ and decrements the index of b, whose binder has disappeared.

4.7. Using the isomorphism

An isomorphism of reduction systems transports every property that is stated in terms of the reduction relation. The de Bruijn development we build on proves the Church–Rosser theorem for $\to_{\mathrm{d}}$ [LEAN]. With Theorem 4.6 it becomes a theorem about named terms up to α-equivalence, with no further work on the named side.

Corollary 4.17 (Church–Rosser for Λ_α)**.** [LEAN] *The relation $\to_\beta$ on Λ_α is confluent: if $M \twoheadrightarrow_\beta N_1$ and $M \twoheadrightarrow_\beta N_2$ then there is P with $N_1 \twoheadrightarrow_\beta P$ and $N_2 \twoheadrightarrow_\beta P$.*

Corollary 4.18. [LEAN] *If $M \twoheadrightarrow_\beta N_1$ and $M \twoheadrightarrow_\beta N_2$ and both N_1 and N_2 are β-normal, then $N_1 = N_2$.*

The same method applies to η-reduction: f is also an isomorphism between η-reduction on Λ_α and Nipkow's η-reduction on dB [21, Section 4.4] [LEAN].

4.8. Notes on the formalization

- *Atoms.* Sections 2 and 3 are parametric in $\mathbb{A}$. The isomorphism is stated for one concrete type of atoms, the natural numbers wrapped in a structure, for which the two hat maps are the projection and the constructor [LEAN].
- *Permutations.* Norrish and Vestergaard represent a permutation as a list of pairs of strings. The formalization uses finitely supported bijections on atoms [LEAN]. Accordingly inc is not defined by recursion over a list but as the permutation that fixes the atom with number 0 and otherwise acts as π with all numbers shifted by one [LEAN].
- *Substitution on* dB. CSLib does not define nsub by the equations of Definition 4.1, but as a composition of a lifting, the substitution sub and a lowering of indices. The equations of Definition 4.1 are proved as lemmas.
- *The bijection.* f is a bijection in the sense of an explicit pair of mutually inverse functions. Its inverse is obtained from surjectivity by choice and is not computable.
- *Equivariance.* There is no general notion of an equivariant relation in the library used. In Theorem 4.14 equivariance is a hypothesis on R, and it is proved separately for each of the relations to which the theorem is applied.

5. Related work

Dedicated mechanizations of α-equivalence are rare. In his concluding section, Crole reports that he and a student are formalizing his proofs in Isabelle/HOL [19] for a follow-up paper [9, Section 7]. We could not find any published account of it.

Apinis and Ahman [2] define α-equivalence of untyped λ-terms directly by inference rules in the Rocq Prover. They show that it is an equivalence and a congruence, preserves free variables and is compatible with substitution [2, Sections 4 and 6]. Their work concerns one new definition and its use, whereas Theorem 3.4 compares several definitions that the literature itself uses. Berghofer and Urban [7] compare formalizations in Isabelle/HOL based on de Bruijn indices and on nominal names on two case studies, among them the substitution lemma [7, Sections 2 and 3]. Crole describes this as comparing definitions of α-equivalence through their applications [9, Section 7], whereas Theorem 3.4 compares them directly.

Norrish and Vestergaard mechanized the isomorphism of Section 4 in HOL4 [21]. Figure 1 shows the further relationships between formalizations of the λ-calculus that they list. Earlier, Gordon [14] built name-carrying terms in HOL on top of de Bruijn terms, so that equality is α-conversion by construction. No relation on raw named terms is involved. Apinis and Ahman [2, Section 11] also define α-equivalence of named terms through a translation to de Bruijn terms and compare the proof obligations with those of their direct relation.

Many other approaches to variable binding have been proposed. The POPLmark challenge [4] posed a set of benchmark problems to compare them in mechanized metatheory. Nominal techniques [23], implemented

in Nominal Isabelle [27], reason about binders through permutations and freshness. Locally nameless representations [3] use names for free variables and de Bruijn indices for bound ones. CSLib contains a locally nameless representation as well [15, Section 5.2]. This paper connects two other representations, the named and the de Bruijn one, and proves the relationships between them explicitly (Section 4).

6. Future work

We see three directions in which this work can be continued.

The network of Figure 1. Λ_α is the quotient of the raw terms by $\sim_{\mathrm{p}}$. By Theorem 3.4, Λ_α is equally the quotient by $\sim_{\mathrm{r}}$ and by $\sim_{\mathrm{r}\#}$, the relations of Barendregt [5, Definition 2.1.11] and of Hindley and Seldin [16, Definition 1.17]. Each remaining α-quotient edge of the figure thus reduces to identifying the α-equivalence of a raw calculus with one of these relations. For λ^C, Vestergaard takes the equivalence generated by Curry's rule with a fresh atom [28, Section 1.2]. It has the form of $\sim_{\mathrm{r}\#}$ but uses Curry's substitution instead of that of Definition 2.4, so the two substitutions have to be shown to agree up to α-equivalence. For λ^V, Vestergaard and Brotherston use a substitution that never renames, together with a side condition that excludes capture [29, Figures 1 and 2]. This relation appears to contain $\sim_{\mathrm{r}}$ and to be contained in $\sim_{\mathrm{r}\#}$. By Theorem 3.4 it would then equal all of them, but we have not checked this formally.

The remaining isomorphisms need further representations: Gordon's two-sorted de Bruijn terms dB_2 [14] and Vestergaard's raw calculus λ^V, whose isomorphism with λ^C Vestergaard has mechanized in Coq [21, Section 2.3]. Formalizing these nodes and edges would bring the whole network of Norrish and Vestergaard into one library.

Further equivalence results. Results in the style of Crole, which show that several natural but superficially different definitions of the same notion coincide, are not unique to α-equivalence. We plan analogous formalizations for the SK combinator calculus. We also plan to connect further binding representations, such as CSLib's locally nameless terms and nominal terms, to the named representation by explicit isomorphisms in the style of Section 4.

Teaching. Fabrizio Montesi (University of Southern Denmark) and Alexandre Rademaker (Getulio Vargas Foundation, Rio de Janeiro), both CSLib maintainers, have expressed interest in using the named representation of Section 2 to teach the λ-calculus and to base assignments on it. It suits this purpose because it stays close to textbook presentations. It has terms with named variables and capture-avoiding substitution, and through Theorem 3.4 the definitions of α-equivalence found in the literature [5, 16] are available side by side and proved equivalent. To support such use, we plan documentation and examples aimed at students.

7. Conclusion

We have formalized in Lean 4, as part of CSLib, Crole's result that several definitions of α-equivalence coincide. We found no errors in Crole's proofs, and the formalization supplies the arguments he omits (Table 2). Following Norrish and Vestergaard, we have also formalized an isomorphism between the quotient of named terms by α-equivalence and pure de Bruijn terms, with their respective notions of β-reduction. Through it, the Church–Rosser theorem for de Bruijn terms yields the Church–Rosser theorem for named terms modulo α-equivalence (Corollary 4.17). Together these results connect two of the standard approaches to variable binding within a single library. As part of CSLib, they are reviewed and kept up to date with Mathlib, so that later work can depend on them directly. They can be extended by further binding representations and by the remaining relationships of the network of Norrish and Vestergaard (Figure 1).

Data availability

The Lean 4 formalization discussed in this paper is part of CSLib (`https://github.com/leanprover/cslib`), released under the Apache 2.0 license. The links in this paper point to a fixed revision at `https://github.com/chris-anto-froeschl/cslib`, built with Lean `v4.36.0-rc1` and Mathlib commit `625d891`. A blueprint of both formalizations, with a dependency graph showing their connections, is at `https://chris-anto-froeschl.github.io/untyped-lambda-calculus/`.

CRediT authorship contribution statement

Chris Anto Fröschl: Conceptualization, Methodology, Software, Formal analysis, Investigation, Writing – original draft, Visualization. **Matthias Güdemann:** Conceptualization, Supervision, Writing – review & editing.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) in order to transcribe mathematical notation from the PDF versions of the cited papers into LaTeX, to check the text for errors and grammar, and to reconstruct Figure 1 from [21, Figure 1]. All transcriptions were checked against the originals. AI assistance also helped to locate less visible sources: the HOL4 development behind the proof pearl [21], and earlier sources for the lemmas of Section 6 of [9]. All of them were checked by the authors against the original texts. The authors take full responsibility for the content of the published article.

Acknowledgements

This work builds on developments that became available in CSLib at the right time. We thank Haoxuan Yin and Fabrizio Montesi for the named representation of the λ-calculus on which Sections 2 and 3 build, Zayn Wang and Maksym Radziwill for the de Bruijn terms and their Church–Rosser proof, and Bas Spitters for the atoms and finite permutations of nominal techniques, both used in Section 4. We also thank Thomas Krishna Waring and Haoxuan Yin for their rigorous reviews of the pull requests, which greatly improved the quality of the code.

References

[1] Achim, T., Best, A., Bietti, A., Der, K., Fédérico, M., Gukov, S., Halpern-Leistner, D., Henningsgard, K., Kudryashov, Y., Meiburg, A., Michelsen, M., Patterson, R., Rodriguez, E., Scharff, L., Shanker, V., Sicca, V., Sowrirajan, H., Swope, A., Tamas, M., Tenev, V., Thomm, J., Williams, H., Wu, L., 2025. Aristotle: IMO-level automated theorem proving. doi:10.48550/arXiv.2510.01346, arXiv:2510.01346.

[2] Apinis, K., Ahman, D., 2025. A simple formalization of alpha-equivalence. doi:10.48550/arXiv.2507.10181, arXiv:2507.10181. version 5.

[3] Aydemir, B., Charguéraud, A., Pierce, B.C., Pollack, R., Weirich, S., 2008. Engineering formal metatheory, in: Proceedings of the 35th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '08), Association for Computing Machinery, New York, NY, USA. pp. 3–15. doi:10.1145/1328438.1328443.

[4] Aydemir, B.E., Bohannon, A., Fairbairn, M., Foster, J.N., Pierce, B.C., Sewell, P., Vytiniotis, D., Washburn, G., Weirich, S., Zdancewic, S., 2005. Mechanized metatheory for the masses: The PoplMark challenge, in: Hurd, J., Melham, T. (Eds.), Theorem Proving in Higher Order Logics, Springer, Berlin, Heidelberg. pp. 50–65. doi:10.1007/11541868_4.

[5] Barendregt, H.P., 1984. The Lambda Calculus: Its Syntax and Semantics. volume 103 of *Studies in Logic and the Foundations of Mathematics*. Revised ed., North-Holland.

[6] Barrett, C., Chaudhuri, S., Montesi, F., Grundy, J., Kohli, P., de Moura, L., Rademaker, A., Yingchareonthawornchai, S., 2026. CSLib: The Lean computer science library. doi:10.48550/arXiv.2602.04846, arXiv:2602.04846.

[7] Berghofer, S., Urban, C., 2007. A head-to-head comparison of de Bruijn indices and names. Electronic Notes in Theoretical Computer Science 174, 53–67. doi:10.1016/j.entcs.2007.01.018.

[8] de Bruijn, N.G., 1972. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem. Indagationes Mathematicae (Proceedings) 75, 381–392. doi:10.1016/1385-7258(72)90034-0.

[9] Crole, R.L., 2012. Alpha equivalence equalities. Theoretical Computer Science 433, 1–19. doi:10.1016/j.tcs.2012.01.030.

[10] Curry, H.B., Feys, R., 1958. Combinatory Logic. volume I. North-Holland, Amsterdam.

[11] Gabbay, M.J., 2011. Foundations of nominal techniques: logic and semantics of variables in abstract syntax. Bulletin of Symbolic Logic 17, 161–229. doi:10.2178/bsl/1305810911. numbering follows the author version, https://www.gabbay.org.uk/papers/fountl.pdf.

[12] Gabbay, M.J., Mathijssen, A., 2008. Capture-avoiding substitution as a nominal algebra. Formal Aspects of Computing 20, 451–479. doi:10.1007/s00165-007-0056-1.

[13] Gabbay, M.J., Pitts, A.M., 2002. A new approach to abstract syntax with variable binding. Formal Aspects of Computing 13, 341–363. doi:10.1007/s001650200016.

[14] Gordon, A.D., 1994. A mechanisation of name-carrying syntax up to alpha-conversion, in: Joyce, J.J., Seger, C.J.H. (Eds.), Higher Order Logic Theorem Proving and Its Applications, Springer, Berlin, Heidelberg. pp. 413–425. doi:10.1007/3-540-57826-9_152.

[15] Henson, C., Montesi, F., 2026. Computer science as infrastructure: the spine of the Lean computer science library (CSLib). doi:10.48550/arXiv.2602.15078, arXiv:2602.15078.

[16] Hindley, J.R., Seldin, J.P., 2008. Lambda-Calculus and Combinators: An Introduction. Second ed., Cambridge University Press. doi:10.1017/CBO9780511809835.

[17] de Moura, L., Ullrich, S., 2021. The Lean 4 theorem prover and programming language, in: Platzer, A., Sutcliffe, G. (Eds.), Automated Deduction – CADE 28, Springer, Cham. pp. 625–635. doi:10.1007/978-3-030-79876-5_37.

[18] Nipkow, T., 2001. More Church–Rosser proofs (in Isabelle/HOL). Journal of Automated Reasoning 26, 51–66. doi:10.1023/A:1006496715975.

[19] Nipkow, T., Wenzel, M., Paulson, L.C., 2002. Isabelle/HOL: A Proof Assistant for Higher-Order Logic. volume 2283 of *Lecture Notes in Computer Science*. Springer, Berlin, Heidelberg. doi:10.1007/3-540-45949-9.

[20] Norrish, M., 2006. Mechanising λ-calculus using a classical first order theory of terms with permutations. Higher-Order and Symbolic Computation 19, 169–195. doi:10.1007/s10990-006-8745-7.

[21] Norrish, M., Vestergaard, R., 2007. Proof pearl: De Bruijn terms really do work, in: Schneider, K., Brandt, J. (Eds.), Theorem Proving in Higher Order Logics, Springer, Berlin, Heidelberg. pp. 207–222. doi:10.1007/978-3-540-74591-4_16.

[22] Pitts, A.M., 2006. Alpha-structural recursion and induction. Journal of the ACM 53, 459–506. doi:10.1145/1147954.1147961.

[23] Pitts, A.M., 2013. Nominal Sets: Names and Symmetry in Computer Science. volume 57 of *Cambridge Tracts in Theoretical Computer Science*. Cambridge University Press. doi:10.1017/CBO9781139084673.

[24] Slind, K., Norrish, M., 2008. A brief overview of HOL4, in: Ait Mohamed, O., Muñoz, C., Tahar, S. (Eds.), Theorem Proving in Higher Order Logics, Springer, Berlin, Heidelberg. pp. 28–32. doi:10.1007/978-3-540-71067-7_6.

[25] The mathlib Community, 2020. The Lean mathematical library, in: Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs, ACM, New York, NY, USA. pp. 367–381. doi:10.1145/3372885.3373824.

[26] Urban, C., Pitts, A.M., Gabbay, M.J., 2004. Nominal unification. Theoretical Computer Science 323, 473–497. doi:10.1016/j.tcs.2004.06.016.

[27] Urban, C., Tasson, C., 2005. Nominal techniques in Isabelle/HOL, in: Nieuwenhuis, R. (Ed.), Automated Deduction – CADE-20, Springer, Berlin, Heidelberg. pp. 38–53. doi:10.1007/11532231_4.

[28] Vestergaard, R., 2003. Structural induction and the λ-calculus. RIMS Kôkyûroku 1318, 30–45. URL: https://www.kurims.kyoto-u.ac.jp/~kyodo/kokyuroku/contents/pdf/1318-5.pdf.

[29] Vestergaard, R., Brotherston, J., 2003. A formalised first-order confluence proof for the λ-calculus using one-sorted variable names. Information and Computation 183, 212–244. doi:10.1016/S0890-5401(03)00023-3.